

Technical Documentation for 机器-教师: the Telegram Bot for learning Chinese

Educational project for learning Chinese vocabulary by HSK level

Document Attribute	Value
Document version	2.0 (Updated for codebase as of June 11, 2026)
Preparation date	June 8, 2026
Technology stack	Python 3.12, python-telegram-bot 22.7, SQLite, jieba

Contents

1. Introduction
2. Functional Requirements
3. Performance Requirements
4. Program Description
5. Software Specification
6. Technical Guides
7. Technical Usage Recommendations
8. Appendix

1. Introduction

1.1. Purpose of This Document

This document defines the purpose, functions, architecture, data structures, deployment procedure, and maintenance rules of the ChinesePythonBot Telegram bot. It is intended for developers, system administrators, and project stakeholders.

The document records the actual state of the source code reviewed on June 11, 2026. It also identifies operational requirements and implementation limitations that should be considered during further development.

1.2. Brief Bot Description

ChinesePythonBot is a Telegram bot designed to help users expand their Chinese vocabulary. A user selects an HSK proficiency level and submits Chinese text. The bot tokenizes the text, detects vocabulary above the selected HSK level or vocabulary available in the local CC-CEDICT dictionary, creates a learning queue, converts confirmed words into learned words, and schedules spaced reviews. The bot also provides statistics, a personal dictionary, and five vocabulary games (translation quiz, scrambled characters, pinyin multiple choice, word chain, and pinyin tone training).

2. Functional Requirements

2.1. Commands and User Functions

Command	Required Behavior	Implementation
/start	Display a welcome message, current HSK level, usage hints, and the main keyboard.	Implemented
/level <1-6>	Validate and save an HSK level from 1 to 6.	Implemented
/add_text	Wait for one text message and analyze its Chinese vocabulary.	Implemented
/learn	Display the next queued word and provide Known and Unknown actions.	Implemented

Command	Required Behavior	Implementation
/review	Start a review session for words whose review date has arrived.	Implemented
/stats	Display HSK level, learned word count, and queue size.	Implemented
/cancel	Cancel the current /add_text conversation.	Implemented
/mydict	Display up to 30 learned words with translations.	Implemented
/game	Open translation, character scramble, and pinyin games.	Implemented
/help	Display the command reference.	Implemented
/reset	Request confirmation and clear learned words and the queue.	Implemented

2.2. New Word Detection by HSK Level

1. The submitted text is segmented with `jieba.lcut()`.
2. Tokens listed in `hsk_data/stopwords.txt` and tokens shorter than two characters are removed.
3. Words already present in the user's `known_words` table are skipped.
4. If a word exists in the combined HSK dictionary, it is considered new only when its HSK level is strictly greater than the user's selected level.
5. If a word does not exist in the HSK dictionary, the bot searches the local CC-CEDICT dictionary. A matching word is added with HSK level 0.
6. Words that have no dictionary match are skipped.
7. A word cannot be duplicated in the queue or learned-word collection for the same user.

Current limitation: Single-character Chinese words are excluded because `tokenizer.py` only keeps tokens with a length of at least two characters. This behavior omits part of the basic HSK vocabulary.

2.3. Learning Queue Logic

- The queue is stored in `words_queue` and is separated by Telegram `user_id`.
- FIFO order is based on `added_at`; the earliest row is selected first.
- Selecting Known removes the word from `words_queue` and inserts it into `known_words`.
- Selecting Unknown updates `added_at` to the current timestamp and moves the word to the end of the queue.
- Each card contains the word, pinyin, translation, HSK level, and source-text context.
- The queue is persistent across bot restarts.

2.4. Spaced Repetition Logic

When a word becomes learned, its first review is scheduled for the following day. The `/review` command selects up to 10 words whose `next_review` date is not later than the current date, ordered by `next_review`.

Answer	Successful Review Number	Base Interval	Scheduling Rule
Known	1	1 day	$\text{base} \times \text{forgetting_curve} \times (5 / \text{difficulty})$
Known	2	3 days	$\text{base} \times \text{forgetting_curve} \times (5 / \text{difficulty})$

Answer	Successful Review Number	Base Interval	Scheduling Rule
Known	3	7 days	$\text{base} \times \text{forgetting_curve} \times (5 / \text{difficulty})$
Known	4 or later	14 days	$\text{base} \times \text{forgetting_curve} \times (5 / \text{difficulty})$
Unknown	Unchanged	1 day	schedule for the following day

The calculated interval is limited to 1 through 90 days. New learned words use difficulty = 3 and forgetting_curve = 0.85. The user interface does not allow these values to be changed.

2.5. Input Requirements

Input	Requirement	Invalid Input Handling
/level	One integer argument from 1 to 6.	The bot returns the required syntax.
/add_text	One non-empty text message that is not a command.	Empty input is rejected.
Chinese text	UTF-8 text in one Telegram message; practical maximum is 4096 characters.	Unknown tokens are skipped.
Callback buttons	Callback data must be generated by the bot.	Expired game state returns an error message.
TOKEN	A valid non-empty TOKEN environment variable.	Application startup fails without a valid token.

3. Performance Requirements

3.1. Target Performance

Metric	Requirement	Notes
Standard commands	Response within 2 seconds under normal Telegram network conditions.	Applies to /start, /level, /stats, and /learn.
Text analysis	Response within 2 seconds for up to 4096 characters on the recommended configuration.	Depends on token count and SQLite operations.
/add_text length	Up to 4096 characters in one message.	No explicit application-level check is implemented.
Review session	Load no more than 10 due words per session.	Enforced by SQL LIMIT 10.

Metric	Requirement	Notes
Stored word count	No fixed application limit.	Limited by SQLite, disk capacity, and query performance.
Concurrency	Suitable for educational or low-load use.	Synchronous sqlite3 calls run inside asynchronous handlers.

3.2. Minimum Resource Requirements

- Operating system: Windows, Linux, or a cloud runtime supporting Python 3.12.
- Processor: 1 virtual CPU core.
- Memory: 256 MB minimum; 512 MB recommended because CC-CEDICT is loaded into memory.
- Disk: at least 100 MB plus capacity for user_data.db growth.
- Network: outbound HTTPS access to the Telegram Bot API.
- Storage: a writable local directory or persistent /app/data volume in cloud deployment.

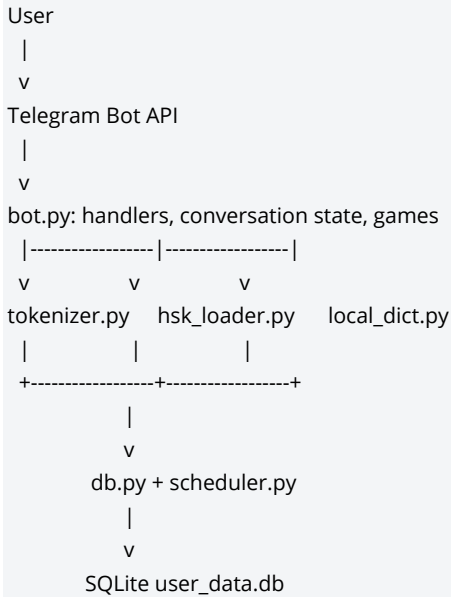
Verification status: The project contains no performance tests or response-time metrics. The values in this section are target requirements and require load testing for formal confirmation.

4. Program Description

4.1. Logical Structure

Subsystem	Implementation	Responsibility
Command handler	bot.py	Telegram handlers, conversations, cards, callbacks, and games.
HSK analyzer	hsk_loader.py, config.py, hsk_data/*.json	Load HSK 1-6 dictionaries and map words to levels.
Tokenizer	tokenizer.py	Segment text with jieba and filter stop words.
Local dictionary	local_dict.py and CC-CEDICT	Provide pinyin and translation for non-HSK words.
Queue and learned words	db.py	SQLite operations, FIFO queue, statistics, and user dictionary.
Spaced repetition	scheduler.py and db.py	Calculate and store the next review date.
Database	user_data.db	Store users, known_words, and words_queue.

4.2. Component Interaction Diagram



4.3. Program Operation

1. main() initializes the SQLite database, builds the Telegram Application, and registers handlers.
2. The application receives updates through long polling with app.run_polling().
3. Telegram routes each update to a command, message, or callback handler.
4. The handler reads user_id and accesses temporary state through context.user_data when required.
5. The handler calls the tokenizer, HSK dictionary, local dictionary, database layer, or repetition scheduler.
6. Results are stored in SQLite and returned to the user as formatted Telegram messages.

4.4. Step-by-Step /add_text Processing

1. The user sends /add_text.
2. ConversationHandler enters TEXT_INPUT and asks for one text message.
3. The next non-empty, non-command message is passed to add_text_process().
4. segment_chinese() applies jieba segmentation and filters stop words and one-character tokens.
5. Each token is checked against known_words.
6. The token is searched in HSK_DICT. A word above the user's HSK level is added to words_queue.
7. If the word is absent from HSK_DICT, LOCAL_DICT is queried and a match is added with hsk_level = 0.
8. The bot stores the sentence containing the word, limited to 150 characters, or the first 100 text characters as fallback.
9. The bot displays the number of new, queued, and learned words and closes the conversation.

4.5. Information and Data Structures

Word Record Format

Field	Type	Purpose
word	TEXT / string	Chinese word and part of the composite primary key.
pinyin	TEXT / string	Pinyin pronunciation.
translation	TEXT / string	Russian translation or CC-CEDICT translation.
hsk_level	INTEGER	HSK level 1-6; 0 indicates a non-HSK dictionary word.
context	TEXT	Source-text fragment containing the word.

Database Schema

Table	Primary Key	Main Fields	Purpose
users	user_id	hsk_level, created_at	Store the user's selected HSK level.
words_queue	(user_id, word)	pinyin, translation, hsk_level, context, added_at	Store new words in FIFO order.
known_words	(user_id, word)	word data, last_reviewed, difficulty, forgetting_curve, next_review, review_count	Store learned words and review scheduling data.

Reset inconsistency: The /reset confirmation states that the HSK level will return to 1, but reset_user_data() only deletes known_words and words_queue. The users row is not updated, so the selected level remains unchanged.

5. Software Specification

No.	File or Component	Purpose
1	bot.py	Application entry point, command handlers, callbacks, conversations, and games.
2	config.py	TOKEN lookup, HSK file paths, and stop-word loading.
3	db.py	SQLite schema and operations for users, queue, learned words, and reviews.
4	hsk_loader.py	Load and combine HSK JSON dictionaries.
5	tokenizer.py	Segment Chinese text with jieba.

No.	File or Component	Purpose
6	scheduler.py	Calculate the next spaced-repetition date.
7	local_dict.py	Load and query the local CC-CEDICT dictionary.
8	hsk_data/hsk1_json ... hsk6_json	HSK 1-6 dictionaries containing 500, 770, 971, 1000, 1071, and 1140 entries.
9	hsk_data/stopwords.txt	43 stop words excluded from tokenization results.
10	hsk_data/cedict_1_0_ts_utf-8_mdbg.txt	Local CC-CEDICT dictionary with approximately 125,000 lines.
11	requirements.txt	Python package versions.
12	runtime.txt	Cloud runtime version: python-3.12.
13	Procfile	Worker command: python bot.py.
14	README.md	Short user-oriented project description.
15	user_data.db	SQLite database created at runtime (path respects /app/data).

6. Technical Guides

6.1. Installation and Startup Requirements

- Python 3.10 or later is required.
- python-telegram-bot 22.7.
- jieba 0.42.1.
- pypinyin 0.55.0.
- sqlite3 is provided by the Python standard library.

Steps:

1. Open a terminal in the directory containing bot.py and hsk_data.
2. Create and activate a virtual environment.
3. Install packages from requirements.txt.
4. Create a Telegram bot with @BotFather and obtain its token.
5. Set the TOKEN environment variable.
6. Run bot.py from the project root.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
$env:TOKEN = "<telegram-bot-token>"
python bot.py
```

On Linux or macOS, activate the environment with `source .venv/bin/activate` and set the token with `export TOKEN='<telegram-bot-token>'`.

6.2. Bot Token Configuration

- Open the official @BotFather account in Telegram.
- Run `/newbot` and store the issued token securely.
- Do not place the token in `config.py`, `README.md`, or source control.
- Use the hosting platform's secrets or configuration-variable service for cloud deployment.
- Revoke and regenerate the token immediately if it is exposed.

6.3. HSK Dictionary Configuration

HSK dictionary paths are defined in `HSK_FILES` in `config.py`. Each UTF-8 JSON file contains an object with a key such (e.g., "HSK1") and an array of records containing word, pinyin, and translation.

- Update `HSK_FILES` when a dictionary file is renamed or moved.
- word field is required; missing pinyin or translation values reduce card quality.
- If the same word occurs at multiple levels, the value loaded from the later file replaces the earlier value.
- Restart the bot after editing dictionary files because `HSK_DICT` is built during module import.

6.4. Database Administration

The local database is `user_data.db` in the current working directory. If `/app/data` exists, the bot uses `/app/data/user_data.db`. Cloud deployments should mount `/app/data` as persistent storage.

```
sqlite3 user_data.db
.tables
.schema users
.schema known_words
.schema words_queue
SELECT COUNT(*) FROM known_words;
.quit
```

Stop the bot and back up `user_data.db` before manual database modification. The current project does not include a migration system.

6.5. Logging and Error Inspection

- Logs are written to `stdout` and `stderr` at `INFO` level.
- Local errors are visible in the terminal running `python bot.py`.
- Cloud errors are available in the worker-process logs of the hosting platform.
- The project has no dedicated log file, rotation, or centralized monitoring.
- Debug `print()` statements in `db.py` should be replaced with `logger.debug()` for production use.

6.6. Post-Deployment Check

1. Confirm that the log contains the bot-started message.
2. Run `/start` and `/level 1` in Telegram.
3. Run `/add_text` and send a Chinese text sample.
4. Verify `/stats` and `/learn`.
5. Mark a word as known and confirm that `known_words` contains the row.
6. Verify that the word becomes available through `/review` on its scheduled date.
7. Test one of the five games via `/game`.

7. Technical Usage Recommendations

7.1. Recommendations for Linguists and Stakeholders

For normal use, refer to the in-bot `/help` command. The correct `/level` value must be set before text analysis because it controls HSK word selection. Words outside the HSK lists may be added with level 0 when they are found in CC-CEDICT.

7.2. Adding New HSK Levels

1. Prepare a UTF-8 JSON dictionary containing word, pinyin, and translation fields.
2. Add the file path to `HSK_FILES` in `config.py`.
3. If a level above 6 is introduced, extend `/level` validation and user-facing messages.
4. Check for duplicate words across levels.
5. Restart the bot and test selection relative to the user's level.

7.3. Changing Review Intervals

Base intervals are defined in `calculate_next_review()` in `scheduler.py`. Modify the 1, 3, 7, and 14 day values, the coefficient formula, or the 90 day limit as required. Add unit tests for successful and unsuccessful reviews, boundary coefficient values, and the maximum interval.

7.4. Supporting Another Language

- Replace `segment_chinese()` with a tokenizer suitable for the target language.
- Replace `HSK_DICT` with a proficiency-level or frequency dictionary.
- Implement a dictionary adapter in place of `ChineseDictionary` and `CC-CEDICT`.
- Generalize pinyin and `hsk_level` to pronunciation and level fields.
- Update card text, games, and context extraction rules.
- Preserve the `db.py` interface or provide a database migration.

8. Appendix

8.1. Example HSK Dictionary Content

```
{
  "HSK1": [
    {"id": 1, "word": "爱", "pinyin": "ài", "translation": "to love; love"},
    {"id": 2, "word": "爱好", "pinyin": "àihào", "translation": "hobby"},
    {"id": 3, "word": "八", "pinyin": "bā", "translation": "eight"},
    {"id": 4, "word": "爸爸", "pinyin": "bàba", "translation": "father; dad"},
    {"id": 5, "word": "吧", "pinyin": "ba", "translation": "modal particle"}
  ]
}
```

The example uses English translations for documentation purposes. The source project currently contains Russian translations. The schema itself is language-independent.

8.2. Example Queue JSON

```
{
  "user_id": 123456789,
  "queue": [
    {
      "word": "学习",
      "pinyin": "xuéxí",
      "translation": "to study; to learn",
      "hsk_level": 2,
      "context": "我每天学习中文。",
      "added_at": "2026-06-10T13:20:15.123456"
    }
  ]
}
```

8.3. Source Project Verification (as of June 11, 2026)

- All Python modules pass `python -m py_compile` syntax validation.
- The six HSK dictionaries contain 5,452 records in total.
- The CC-CEDICT file is approximately 9.7 MB and contains approximately 125,000 lines.
- Queue persistence verified (no DROP TABLE on startup).
- Five games implemented and functional.
- The submitted archive contains no automated test suite.